

A PROJECT DOCUMENTATION ON

SWIFT SHIP TRACKER

A Web-Based Shipment Tracking System with Animated Route Map, On-Time
Confidence Score and Tamil/English Support

Submitted by

R Abishek

M Deepak

M R Mohamed Akil

R Raja

K Thavasimani

Academic Year: 2026 - 2027

Abstract

Tracking a parcel normally means typing a number into a plain page and reading a line of status text. Swift Ship Tracker is a web application that makes shipment tracking easier to understand. It shows each shipment on an animated route map, lists every checkpoint on a journey timeline, and gives an on-time confidence score so that the customer knows how likely the parcel is to arrive as promised.

The system supports sea, air and road shipments. A dashboard lists all shipments with colour-coded status (On time, Delayed, Delivered). A user can search by tracking number, simulate the next stop of a shipment to receive a notification, copy a share link, and switch between English and Tamil and between light and dark themes. The project is built using only HTML, CSS and JavaScript, so it runs in any modern browser without installation or a server.

Keywords: shipment tracking, logistics, route visualisation, ETA confidence, HTML, CSS, JavaScript.

Table of Contents

Certificate, Declaration, Acknowledgement, Abstract

1. Introduction

2. Objectives

3. System Analysis (existing system, proposed system, feasibility study)

4. Technologies Used

5. System Requirements

6. System Design (architecture, use case, data flow diagrams, flowchart, data structure, algorithm, UI design)

7. Implementation

8. Testing

9. User Manual

10. Output Screens

11. Limitations and Future Scope

12. Conclusion

References

Appendix A: Source Code

Appendix B: Sample Data and Glossary

List of Figures

Figure 6.1: System architecture

Figure 6.2: Use case diagram

Figure 6.3: Data flow diagram, level 0

Figure 6.4: Data flow diagram, level 1

Figure 6.5: Flowchart of the tracking process

Figures 10.1 to 10.6: Output screens

1. Introduction

1.1 Background

Online shopping and international trade have increased the number of parcels moving by sea, air and road every day. Customers want to know where their parcel is and when it will arrive. Most tracking pages show only a list of status messages, which is hard to follow, especially for shipments that pass through several ports or hubs.

1.2 Problem Statement

Existing tracking pages give status text but do not show the journey visually, do not tell the customer how reliable the delivery date is, and are usually available in one language only. A customer with several parcels must also open each tracking page separately.

1.3 Motivation

A parcel passing through Chennai Port, a transit port, a hub and a final city goes through many checkpoints. Showing these checkpoints on a map and giving a simple percentage for on-time delivery helps ordinary users understand the situation at a glance. Offering the page in Tamil makes it easier for local users.

1.4 Scope

This project is a front-end prototype. It demonstrates the user interface and the logic of a tracking system using sample shipment data stored in JavaScript. It does not connect to a real courier network. The design allows the sample data to be replaced later by a database or a courier API.

1.5 Organisation of the Report

Chapter 2 lists the objectives. Chapter 3 analyses the existing and proposed systems. Chapter 4 describes the technologies. Chapter 5 gives the requirements. Chapter 6 explains the design, and Chapter 7 the implementation. Chapter 8 covers testing, Chapter 9 is the user manual, and Chapter 10 shows the output screens. Chapters 11 and 12 give the limitations, future scope and conclusion. The complete source code is in Appendix A.

2. Objectives

- To build a simple and attractive web interface for tracking sea, air and road shipments.
- To show the journey of a parcel on an animated route map with a moving vehicle icon.
- To calculate an on-time confidence score for each shipment from the weather risk of the remaining stops.
- To show all shipments on one dashboard with colour-coded status.
- To support Tamil and English so that local users can read the application comfortably.
- To provide dark mode, share link, QR-style code and notification features.
- To build the project with only HTML, CSS and JavaScript so that it is easy to run and explain.

3. System Analysis

3.1 Existing System

- Shows only a text list of statuses.
- No visual route or map animation.
- No indication of how likely the delivery is to be on time.
- One parcel per page; no combined dashboard.
- Usually one language only.

3.2 Proposed System

Swift Ship Tracker overcomes these problems by combining a dashboard, a route map, a timeline and a confidence score in a single page.

Feature	Existing system	Swift Ship Tracker
Route view	Text only	Animated route map
Delivery reliability	Only a date	On-time confidence score (%)
Multiple parcels	Separate pages	Single dashboard
Language	English only	English and Tamil
Sharing	Not available	Share link and QR-style code
Theme	Fixed	Light and dark mode

3.3 Feasibility Study

Technical feasibility: the project uses only standard HTML, CSS and JavaScript that every modern browser supports. No server, database or special library is needed.

Economic feasibility: all tools used (text editor, browser, Google Fonts) are free. There is no hosting cost because the project can run directly from a folder on the computer.

Operational feasibility: the interface is simple and uses familiar elements such as a search box, a list and buttons. Users need no training, and Tamil support helps local users.

4. Technologies Used

Technology	Use in this project
HTML5	Defines the page structure: header, search box, list area, details area and notification area. Semantic tags (header, main, section) and ARIA labels help accessibility.
CSS3	Styles the page. CSS variables store colours, so dark mode needs only a second set of values. CSS Grid builds the two-column layout and media queries make it responsive.
JavaScript (ES6)	Holds the data and all logic: search, calculation, drawing, animation, translation and notifications. Uses arrow functions, template strings, array methods and destructuring.
SVG	Draws the route map (lines, circles, text) directly in the page so that it stays sharp at every screen size.
Google Fonts	Bricolage Grotesque typeface for headings and text, with system fonts as fallback.
Visual Studio Code	Editor used to write the code.

4.1 Why only HTML, CSS and JavaScript?

- The project runs by opening one file; no installation is needed.
- It is easy to demonstrate in a viva or a laboratory computer.
- The same code can later be used as the front end of a Node.js or other back-end system.

5. System Requirements

5.1 Hardware Requirements

Component	Requirement
Processor	Any modern dual-core processor
RAM	2 GB or more
Storage	Less than 1 MB for the project files
Display	Any screen; layout is responsive for mobile phones

5.2 Software Requirements

Component	Requirement
Operating system	Windows, Linux, macOS, Android or iOS
Browser	Chrome, Edge, Firefox or Safari (recent version)
Languages	HTML5, CSS3, JavaScript (ES6)
Editor	Visual Studio Code or any text editor
Internet	Optional; needed only to load the Google Font

5.3 Functional Requirements

- FR1: The system shall list all shipments with their status.
- FR2: The user shall be able to search a shipment by tracking number; the search is not case sensitive.
- FR3: The system shall show the route, current position and checkpoint timeline of the selected shipment.
- FR4: The system shall calculate and show the on-time confidence score.
- FR5: The user shall be able to simulate the next stop and receive a notification.
- FR6: The user shall be able to copy a share link and see a QR-style code.
- FR7: The user shall be able to switch language between English and Tamil.
- FR8: The user shall be able to switch between light and dark theme.

5.4 Non-Functional Requirements

Requirement	Description
Usability	Clear labels, large buttons, colour-coded status and a simple one-page layout.
Performance	Page loads instantly because there is no server request; animation takes about 0.9 seconds.

Requirement	Description
Portability	Runs on any device with a modern browser.
Maintainability	Code is split into three files; shipment data is kept in one array.
Accessibility	Visible keyboard focus, labels for buttons and map, and reduced-motion support.
Responsiveness	Layout stacks into one column below 760 px width.

6. System Design

6.1 System Architecture

The application is a single-page front-end application. The browser loads index.html, which links style.css and script.js. All data is held in JavaScript objects and the page is redrawn by JavaScript functions whenever the user selects a shipment, changes language, or moves a shipment to its next stop.

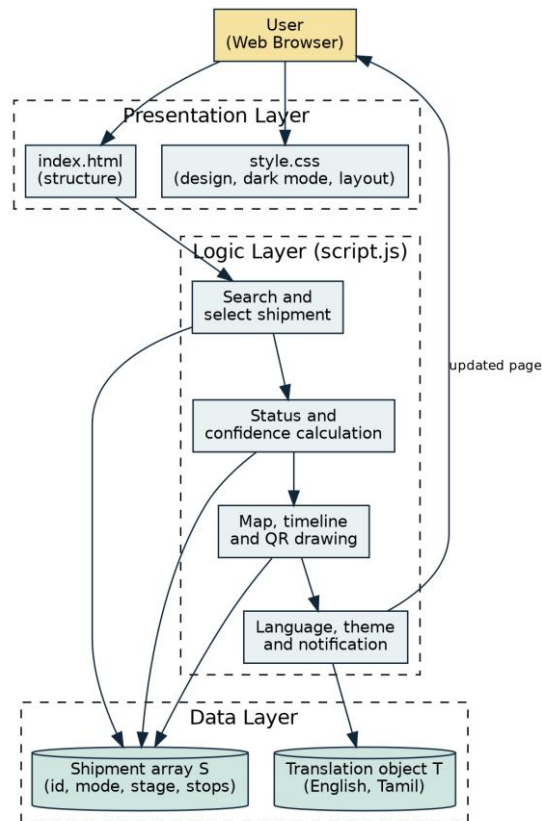


Figure 6.1: System architecture

6.2 Use Case Diagram

The only actor is the user (customer). The use cases are the actions that the user can perform in the application.

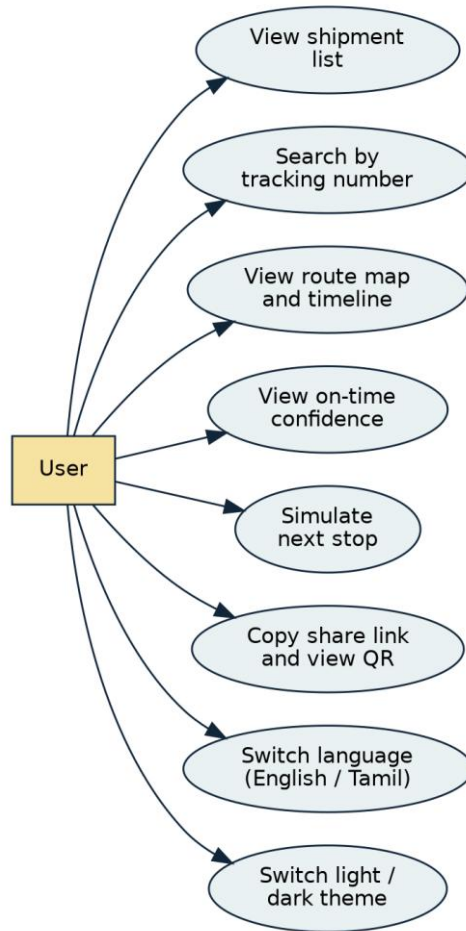


Figure 6.2: Use case diagram

6.3 Data Flow Diagrams

The level 0 diagram shows the whole system as one process. The user gives a tracking number and button actions; the system returns status, map, timeline, confidence and notifications.

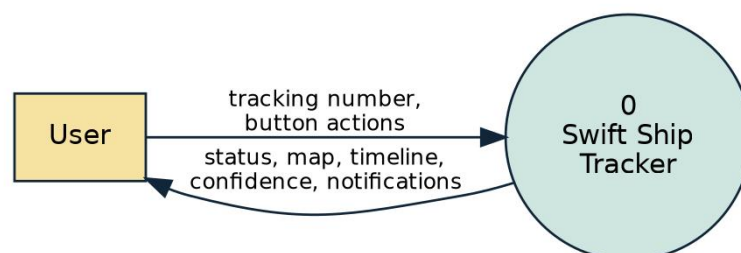


Figure 6.3: Data flow diagram, level 0

The level 1 diagram splits the system into four processes: search or select, calculate status and confidence, draw map and timeline, and simulate next stop. All of them use the shipment data store D1.

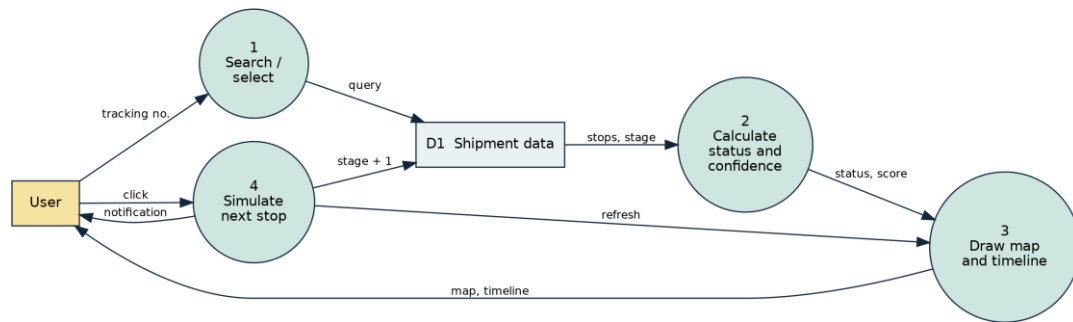


Figure 6.4: Data flow diagram, level 1

6.4 Flowchart

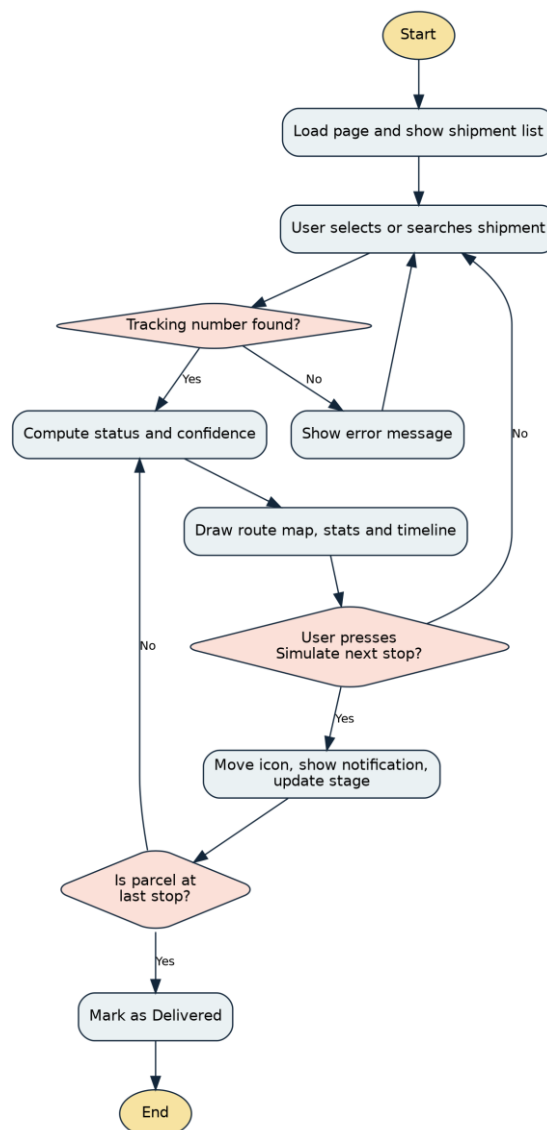


Figure 6.5: Flowchart of the tracking process

6.5 Modules

Module	Description
Dashboard	Lists shipments with a colour strip: yellow for on time, red for delayed, green for delivered.
Search	Finds a shipment by tracking number and shows an error message if it is not found.
Route map	Draws the route as an SVG with stops, a red line for the distance covered, and a moving vehicle icon.
Confidence	Calculates the on-time confidence percentage.
Timeline	Lists every checkpoint as done, current or upcoming, with the weather risk.
Notification	Shows a pop-up message when a shipment reaches a stop or is delivered.
Language and theme	Switches text between English and Tamil and colours between light and dark.
Share	Copies a link to the shipment and shows a QR-style pattern.

6.6 Data Structure

Each shipment is an object with the fields below. Each stop is an array: [name, x, y, weatherRisk], where x and y are the position on the map and weatherRisk is a percentage.

Field	Meaning
id	Tracking number, for example SS1001
name	Description of the goods
mode	sea, air or road
icon	Emoji shown as the moving vehicle
stage	Index of the stop where the parcel is now
stops	List of checkpoints with map position and weather risk
t	Arrival date for each reached stop

6.7 On-Time Confidence Algorithm

The confidence score starts at 100 percent. The weather risk of every stop that is still ahead is subtracted. For the demonstration shipment SS1001, an extra 18 percent penalty represents a known port delay. The score never goes below 40 percent.

$$\text{confidence} = \max(40, 100 - \text{sum}(\text{risk of remaining stops}) - \text{delayPenalty})$$

Status rules: if the last stop is reached, the status is Delivered; otherwise if confidence is below 80 percent it is Delayed; otherwise it is On time.

Example: SS1001 is at Colombo Transit. The three stops still ahead (Jebel Ali, Suez Canal, Rotterdam) have risks 4, 9 and 7 percent. The score is $100 - 20 - 18 = 62$ percent, so the shipment is shown as Delayed.

6.8 User Interface Design

The design uses the look of a nautical chart: a pale sea-chart background, deep navy ink for text, and signal colours taken from port equipment (container red, sea green and buoy yellow). The route map has a light sea-blue background with a wave band at the bottom.

Element	Choice
Light colours	Background #E9EFEF, panel #F7FAF9, ink #12263A
Signal colours	Red #B8432F (delayed), green #2F7F79 (delivered), yellow #C98F0C (on time)
Dark colours	Background #0D1A26, panel #13263A, text #E6EEF1
Typeface	Bricolage Grotesque, fallback system-ui and sans-serif
Layout	Two columns (list 240-320 px, details flexible); one column below 760 px
Motion	Vehicle icon moves for about 0.9 s; notification slides in; both respect reduced-motion settings where applicable

7. Implementation

7.1 Project Structure

File	Purpose
index.html	Page structure: header, search box, shipment list, detail panel, notification area
style.css	Colour variables for light and dark mode, grid layout, buttons, timeline, map and mobile layout
script.js	Shipment data, translations, calculations, drawing functions and event handlers

7.2 HTML

The HTML file contains only the fixed parts of the page. The shipment list and the detail panel are created by JavaScript. The meta viewport tag makes the page fit mobile screens. The notification area has `role="status"` so that screen readers announce its messages.

7.3 CSS

Colours are stored as CSS variables, such as `--bg`, `--ink` and `--red`. Dark mode works by redefining these variables, so no element needs to be styled twice. Dark mode is applied automatically when the device is set to dark, and the Dark/Light button overrides it. CSS Grid places the shipment list beside the detail panel on large screens and stacks them on small screens. The notification is hidden with opacity and moved off-screen, and slides in with a transition.

7.4 JavaScript

The data is kept in two objects: S (the shipments) and T (translations for English and Tamil). The main functions are described below.

Function	Work done
<code>conf(s)</code>	Calculates the on-time confidence of shipment s
<code>status(s)</code>	Returns Delivered, Delayed or On time
<code>col(s)</code>	Returns the colour that matches the status
<code>list()</code>	Draws the shipment list on the left
<code>detail()</code>	Draws the map, statistics, timeline, buttons and QR pattern; attaches the button actions
<code>qr(id)</code>	Creates a QR-style pattern from the tracking number (visual only, not scannable)
<code>toast(msg)</code>	Shows the pop-up notification for about 2.6 seconds

Function	Work done
ui()	Refreshes all text after the language or theme is changed

7.5 Key Code Explained

Confidence and status calculation:

```
const conf=s=>Math.max(40,100-s.stops.slice(s.stage+1)
    .reduce((a,x)=>a+x[3],0)-(s.id==="SS1001"?18:0));
const status=s=>s.stage===s.stops.length-1?"dv":conf(s)<80?"dl":"ot";
```

slice() takes the stops that are still ahead, reduce() adds their weather risks, and Math.max keeps the score at 40 or above. status() then returns dv (delivered), dl (delayed) or ot (on time).

Vehicle animation:

```
const k=Math.min(1,(now-st)/900);
const x=from[1]+(to[1]-from[1])*k, y=from[2]+(to[2]-from[2])*k;
if(k<1) requestAnimationFrame(f);
```

k goes from 0 to 1 in 900 milliseconds. The icon position is the starting stop plus k times the distance to the next stop, so it moves in a straight line.

Language switching:

```
const t=k=>T[L][k];
$("lang").onclick=()=>{L=L==="en"? "ta": "en";ui()};
```

Every visible text is read through t(). Changing L between en and ta and calling ui() redraws the whole page in the other language.

8. Testing

8.1 Test Approach

The application was tested manually by running index.html in a Chromium-based browser (the engine used by Chrome and Edge) on a desktop-size window and a 390 px wide mobile-size window. Each function was checked against its expected result. The browser console showed no JavaScript errors during testing.

8.2 Test Cases

No.	Test case	Expected result	Result
1	Open index.html	Dashboard shows 3 shipments	Pass
2	Search ss3003 (lower case)	Shipment SS3003 is opened	Pass
3	Search XYZ	Message: Tracking number not found	Pass
4	Open SS1001	Status Delayed, confidence 62%	Pass
5	Press Simulate next stop	Icon moves; notification shown; stops left decreases	Pass
6	Move shipment to last stop	Status becomes Delivered; button disabled	Pass
7	Press language button	All text changes to Tamil and back	Pass
8	Press theme button	Colours switch between light and dark	Pass
9	Press Copy share link	Link is copied and confirmation shown; if the browser blocks copying, the link is shown in a box	Pass
10	Reduce window to 600 px	List and details stack in one column	Pass
11	Open index.html#SS3003	Shipment SS3003 is selected on load	Pass

8.3 Issues Found and Fixed

- The route of SS1001 first went to Singapore and then back to Colombo. The stops were re-ordered as Chennai, Colombo, Jebel Ali, Suez Canal and Rotterdam so that the route moves forward.
- Reached stops without a date showed a dash. Dates were added to the data.
- The share button showed "Link copied" even when the browser blocked copying. The code now waits for the result and shows the link in a box if copying fails.

- The hidden notification was visible as a small dark shape in full-page screenshots. It is now fully transparent when hidden.

8.4 Browser Compatibility

The project uses only standard features (CSS Grid, CSS variables, SVG, requestAnimationFrame and the Clipboard API with a fallback). It is therefore expected to work on current versions of Chrome, Edge, Firefox and Safari. Final checking on the browsers available in the college laboratory is recommended before submission.

9. User Manual

9.1 Starting the Application

- Copy the project folder (index.html, style.css, script.js) to the computer.
- Double-click index.html. The application opens in the default browser.
- If the page opens without styling, check that all three files are in the same folder.

9.2 Using the Application

Action	Steps
View a shipment	Click a shipment in the list on the left. Its map, score and timeline appear on the right.
Search by number	Type a tracking number such as SS3003 in the search box and press Enter or click Track.
Read the status colour	Yellow strip: on time. Red strip: delayed. Green strip: delivered.
Read the confidence score	The percentage under the map shows how likely the shipment is to arrive on time.
Simulate movement	Click Simulate next stop. The vehicle moves to the next checkpoint and a notification appears.
Share a shipment	Click Copy share link. Send the link to another person; opening it selects the same shipment.
Change language	Click the language button at the top right (Tamil or English).
Change theme	Click the Dark or Light button at the top right.

9.3 Adding a New Shipment

Open script.js and find the array S at the top. Copy one shipment object, then change the id, name, mode, icon, stage and stops. Save the file and refresh the browser.

10. Output Screens

This chapter shows the output of the running application. Each figure was captured from the project opened in a web browser.

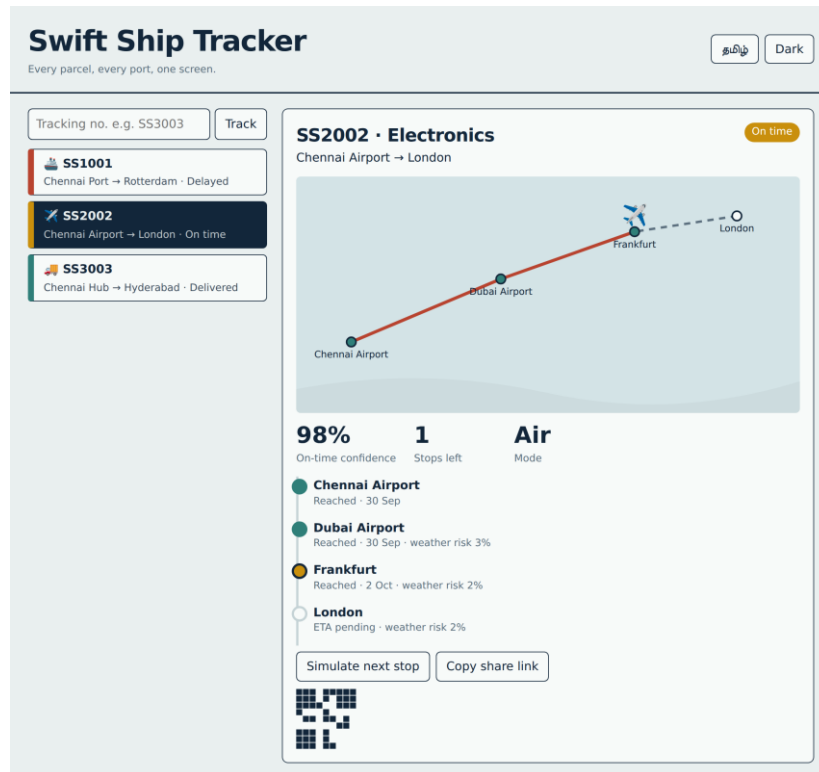


Figure 10.1: Dashboard showing the shipment list, route map and timeline of an on-time air shipment (SS2002)

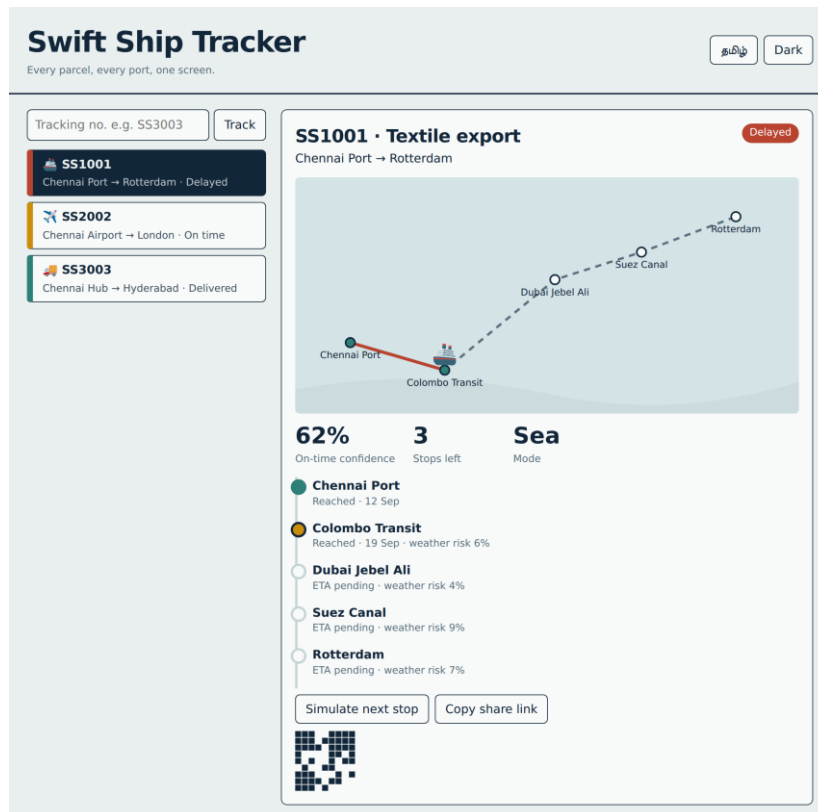


Figure 10.2: Delayed sea shipment (SS1001) with 62% on-time confidence and red status strip

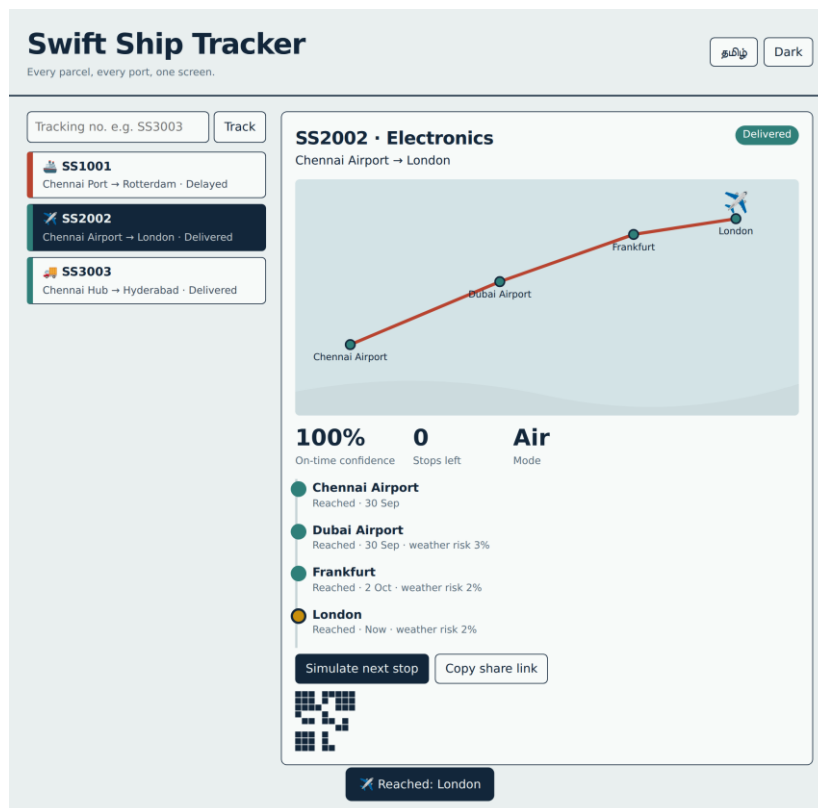


Figure 10.3: Notification shown after pressing Simulate next stop; the shipment is now Delivered

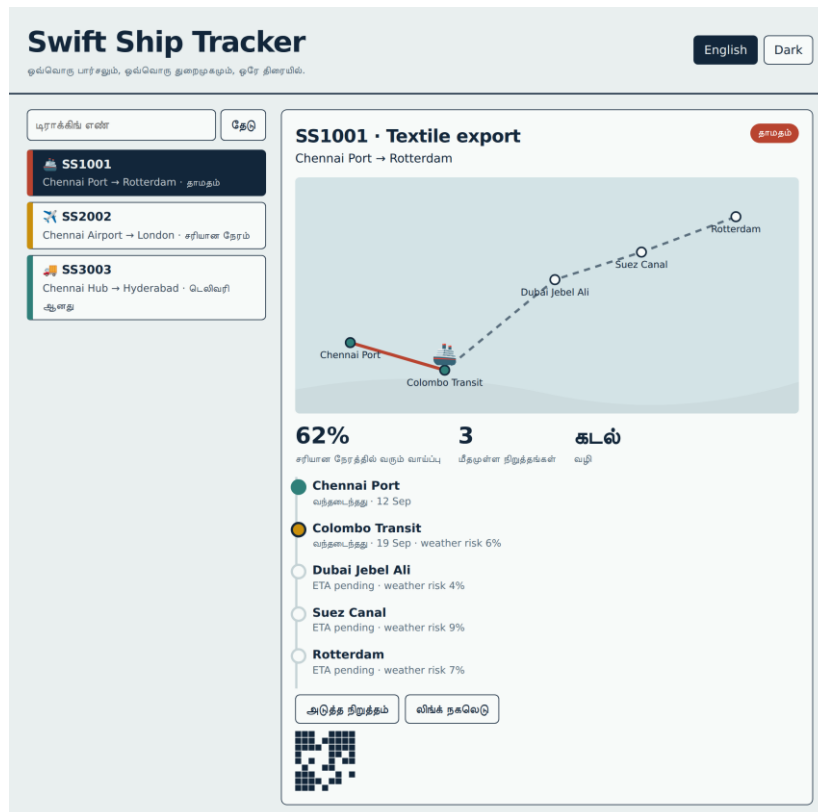


Figure 10.4: Application in Tamil language mode

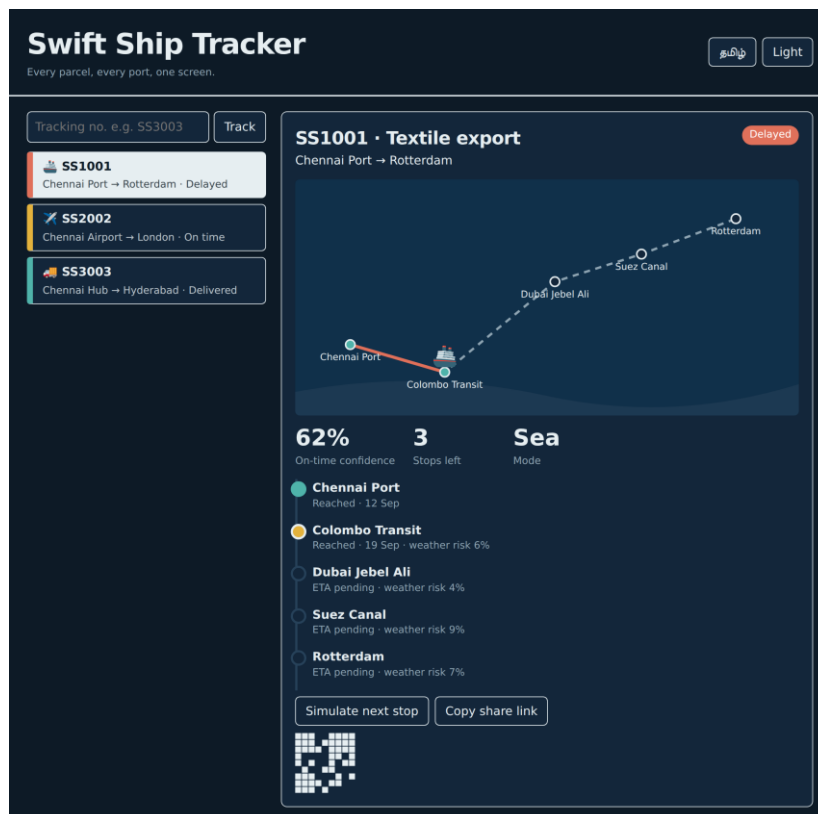


Figure 10.5: Dark mode

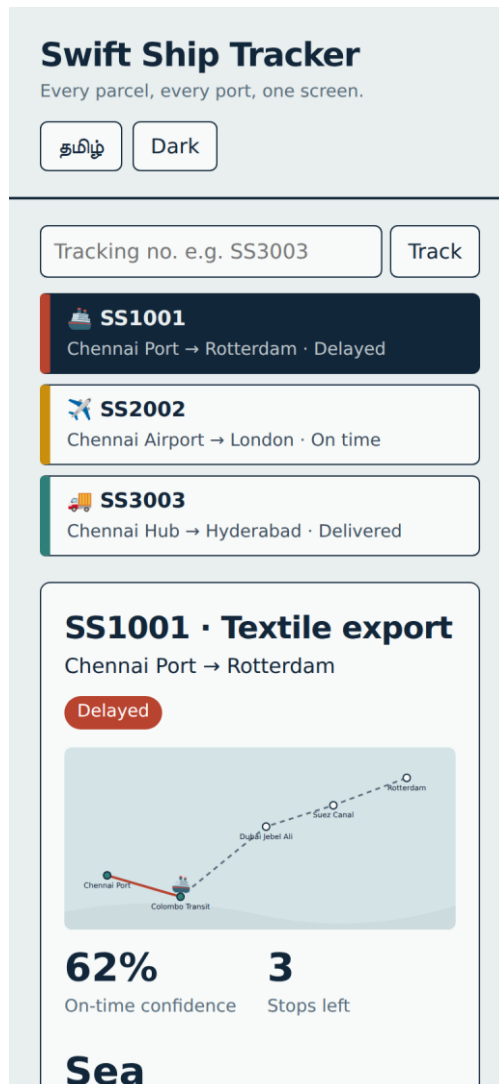


Figure 10.6: Mobile view of the responsive layout

11. Limitations and Future Scope

11.1 Limitations

- Shipment data and weather risk are sample values, not live data.
- The map is a stylised diagram, not a real geographic map.
- The QR-style pattern is visual only and cannot be scanned.
- Shipment progress is not saved when the page is reloaded.
- The extra 18 percent delay penalty for SS1001 is fixed in the code for demonstration.

11.2 Future Scope

- Back end with Node.js and a MongoDB database for real shipment records.
- Real maps using Leaflet or Google Maps.
- Live weather data from a weather API to compute the risk.
- Real QR codes and SMS or email notifications.
- User login and an admin panel for adding shipments.
- A mobile app version.

12. Conclusion

Swift Ship Tracker shows that shipment tracking can be made clearer by combining a dashboard, an animated route, a timeline and an on-time confidence score. The project meets its objectives using only HTML, CSS and JavaScript, supports Tamil and English, works on desktop and mobile, and offers dark mode. With a real database and live data, the same design can be developed into a complete tracking service.

References

- [1] Mozilla Developer Network (MDN) Web Docs, HTML, CSS and JavaScript reference. developer.mozilla.org
- [2] W3C, SVG (Scalable Vector Graphics) specification. [w3.org](https://www.w3.org)
- [3] Google Fonts, Bricolage Grotesque typeface. fonts.google.com
- [4] MDN Web Docs, `window.requestAnimationFrame()`. developer.mozilla.org
- [5] MDN Web Docs, CSS Grid Layout and CSS Custom Properties. developer.mozilla.org
- [6] MDN Web Docs, Clipboard API. developer.mozilla.org

Appendix A: Source Code

The complete source code of the project is given below.

A.1 index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1, viewport-fit=cover">
<title>Swift Ship Tracker</title>
<link
href="https://fonts.googleapis.com/css2?family=Bricolage+Grotesque:wght@400;600;800&display=swap"
rel="stylesheet">
<link rel="stylesheet" href="style.css">
</head>
<body>
<header>
<h1>Swift Ship Tracker<small id="tag"></small></h1>
<div class="tools">
<button id="lang" aria-label="Language">தமிழ்</button>
<button id="theme" aria-label="Theme">Dark</button>
</div>
</header>
<main>
<section>
<div class="search"><input id="q" placeholder="Tracking no. e.g. SS3003" aria-label="Tracking
number"><button id="go"></button></div>
<div class="list" id="list"></div>
</section>
<section class="panel" id="detail"></section>
</main>
<div id="toast" role="status"></div>
<script src="script.js"></script>
</body>
</html>
```

A.2 style.css

```
:root{--bg:#e9efef;--panel:#f7faf9;--ink:#12263a;--mute:#5b6f7e;--line:#c5d3d6;--red:#b8432f;--
green:#2f7f79;--yellow:#c98f0c;--sea:#d3e3e6;
box-sizing:border-box;padding-top:env(safe-area-inset-top,0px);padding-bottom:env(safe-area-inset-
bottom,0px)}
@media (prefers-color-scheme:dark){:root{not([data-theme="light"]){--bg:#0d1a26;--panel:#13263a;--
ink:#e6eef1;--mute:#8fa5b3;--line:#27425a;--red:#e0705a;--green:#4fb3aa;--yellow:#e3b23c;--
sea:#10304a}}
:root[data-theme="dark"]{--bg:#0d1a26;--panel:#13263a;--ink:#e6eef1;--mute:#8fa5b3;--line:#27425a;--
red:#e0705a;--green:#4fb3aa;--yellow:#e3b23c;--sea:#10304a}
html{scroll-padding-top:env(safe-area-inset-top,0px)}
*{box-sizing:border-box}
body{margin:0;background:var(--bg);color:var(--ink);font-family:'Bricolage Grotesque',system-
ui,sans-serif;line-height:1.45}
header{display:flex;flex-wrap:wrap;gap:12px;align-items:center;justify-content:space-
between;padding:20px 24px;border-bottom:2px solid var(--ink)}
h1{margin:0;font-size:clamp(1.6rem,4vw,2.4rem);font-weight:800;letter-spacing:-.03em}
h1 small{display:block;font-size:.85rem;font-weight:400;color:var(--mute);letter-spacing:0}
.tools{display:flex;gap:8px;flex-wrap:wrap}
button,input{font:inherit;color:inherit}
button{background:var(--panel);border:1.5px solid var(--ink);border-radius:6px;padding:7px
13px;cursor:pointer}
button:hover{background:var(--ink);color:var(--bg)}
button:focus-visible,input:focus-visible{outline:3px solid var(--yellow);outline-offset:2px}
main{display:grid;grid-template-columns:minmax(240px,320px) 1fr;gap:20px;padding:20px 24px;max-
width:1100px;margin:auto}
@media (max-width:760px){main{grid-template-columns:1fr}}
.search{display:flex;gap:6px;margin-bottom:12px}
.search input{flex:1;min-width:0;padding:8px 10px;border:1.5px solid var(--ink);border-
radius:6px;background:var(--panel)}
.list{display:flex;flex-direction:column;gap:8px}
.item{text-align:left;display:block;width:100%;border-left:8px solid var(--c);border-radius:4px 6px
6px 4px}
.item.on{background:var(--ink);color:var(--bg)}
.item b{display:block}.item span{font-size:.85rem;opacity:.8}
.panel{background:var(--panel);border:1.5px solid var(--ink);border-radius:8px;padding:18px}
.top{display:flex;flex-wrap:wrap;justify-content:space-between;gap:12px;align-items:flex-start}
.top h2{margin:0;font-size:1.5rem;letter-spacing:-.02em}
.badge{padding:3px 10px;border-radius:99px;color:#fff;font-size:.85rem;background:var(--c)}
.map{width:100%;height:auto;background:var(--sea);border-radius:6px;margin:14px 0;display:block}
.stats{display:flex;gap:24px;flex-wrap:wrap;margin-bottom:10px}
.stats div{min-width:110px}.stats strong{display:block;font-size:1.9rem;line-height:1.1}
.stats span{color:var(--mute);font-size:.85rem}
ol{list-style:none;margin:14px 0 0;padding-left:3px solid var(--line)}
li{position:relative;padding:0 0 14px 20px}
li::before{content:"";position:absolute;left:-9px;top:3px;width:15px;height:15px;border-
radius:50%;background:var(--panel);border:3px solid var(--line)}
li.done::before{background:var(--green);border-color:var(--green)}
li.now::before{background:var(--yellow);border-color:var(--ink)}
li small{display:block;color:var(--mute)}
.actions{display:flex;gap:8px;flex-wrap:wrap;margin-top:8px}
```



```
$("lang").onclick={()=>{L=L=="en"? "ta": "en"; ui()}};
$("theme").onclick={()=>{dark=!dark; document.documentElement.dataset.theme=dark?"dark": "light"; ui()}};
if (dark) document.documentElement.dataset.theme="dark";
const h=location.hash.slice(1); if (S.find(s=>s.id===h)) cur=S.find(s=>s.id===h);
ui();
```

Appendix B: Sample Data and Glossary

B.1 Sample Shipments

ID	Goods	Mode	Route (weather risk %)
SS1001	Textile export	Sea	Chennai Port (0) - Colombo Transit (6) - Dubai Jebel Ali (4) - Suez Canal (9) - Rotterdam (7); now at Colombo Transit
SS2002	Electronics	Air	Chennai Airport (0) - Dubai Airport (3) - Frankfurt (2) - London (2); now at Frankfurt
SS3003	Books and gifts	Road	Chennai Hub (0) - Vellore (1) - Bengaluru (2) - Hyderabad (1); delivered

B.2 Glossary

Term	Meaning
Shipment	A parcel or cargo that is moved from one place to another
Checkpoint / stop	A port, airport or hub through which a shipment passes
ETA	Estimated time of arrival
Confidence score	Percentage chance that the shipment arrives on time, as calculated by this project
SVG	Scalable Vector Graphics, a way to draw shapes in a web page
DOM	Document Object Model, the page structure that JavaScript changes
Responsive design	A layout that adjusts to the screen size